Servidor HTTP/1.1 com Qualidade de Serviço em C com Pthreads: terceira entrega

Giovana Piazza[1], **Luis Felipe Maccalli**[1], **Vitor Moreira**[1]

[1]Universidade Federal do Pampa (Unipampa)
Fundamentos e Avaliação de Redes de Computadores – 2026/2

Abstract. *This report presents the second version of a concurrent, persistent HTTP/1.1 server written in C using POSIX threads. It implements per-IP rate limits, admission control, shared rates among connections and a live terminal panel. Packet captures taken with Wireshark (dumpcap) and traffic statistics from IPTraf-ng on the loopback interface show per-IP throughput within 1% of the configured rates, rate sharing between connections of the same address and admission rejections with HTTP 503. Experiments on physical Ethernet and Wi-Fi networks were not performed.*

Resumo. *Este relatório apresenta a segunda versão de um servidor HTTP/1.1 concorrente e persistente em C com Pthreads. A implementação acrescenta taxas por IP, controle de admissão, divisão da taxa entre conexões e painel em tempo real. Capturas feitas com o Wireshark (dumpcap) e estatísticas do IPTraf-ng na interface de loopback mostram vazão por IP a menos de 1% das taxas configuradas, divisão da taxa entre conexões do mesmo endereço e recusa por controle de admissão com HTTP 503. Experimentos em redes físicas Ethernet e Wi-Fi não foram realizados.*

1. Introdução

A terceira entrega acadêmica, prevista para 19/10/2026, corresponde ao relatório e ao código da segunda versão do software. O enunciado exige servidor HTTP em Linux, C e Pthreads, com clientes concorrentes, conexões persistentes e mecanismos de qualidade de serviço (QoS) [1]. A primeira versão constitui a base de HTTP e concorrência; a segunda inclui controle de envio por IP e monitoramento.

O servidor aplica o ritmo de envio na camada de aplicação, acima do TCP. Isso não substitui o controle de congestionamento ou de fluxo do transporte. A implementação é um subconjunto didático de HTTP/1.1, com GET, HEAD e arquivos estáticos, e não uma implementação completa das RFCs [3, 4].

2. Metodologia e organização

O repositório mantém a primeira versão em V1 e a segunda em V2. A equipe segue Scrum com quadro no Trello e papéis rotativos por sprint. Na Sprint 2 (06/10 a 19/10), a V2 foi implementada sobre a V1, testada com uma suíte automatizada e avaliada com Wireshark e IPTraf-ng. Os registros dos primeiros experimentos estão em `evidencias/v2`, os testes automatizados em `evidencias/revisao-v2` e as capturas da avaliação com as ferramentas em `evidencias/ferramentas-v2`. O cronograma restante prevê a validação com o docente até 26/10 e os slides até 03/12, conforme o enunciado.

3. Desenvolvimento

3.1. Taxas, concorrência e admissão

A execução segue o formato `./server PORTA DIRETORIO VAZAO_MAX_KBPS [ARQUIVO_TAXAS]`. O arquivo auxiliar associa IPv4 e taxa inteira em kbps, usando a convenção decimal: 1 kbps = 1000 bits/s. Endereços não cadastrados usam 1000 kbps, desde que essa reserva caiba na capacidade informada.

Um mutex protege a tabela de IPs, suas conexões e a soma reservada. Se o IP já estiver ativo, a conexão divide a reserva existente; para um IP sem conexões ativas, a admissão exige $V_{reservada} + R_{IP} \leq V_{max}$. A última conexão libera a reserva. Uma recusa gera resposta 503. O limite de 128 conexões também protege os recursos do processo.

O servidor rejeita taxas zero, negativas, inválidas, maiores que 1.000.000.000 kbps, valores fora da representação numérica, IPv4 inválido, IP duplicado e campos excedentes. A capacidade passa pela mesma validação numérica. Comentários com # e linhas vazias são aceitos. Um arquivo explicitamente informado que não pode ser aberto interrompe a inicialização; quando apenas o arquivo padrão não existe, permanece a taxa padrão.

3.2. Envio e divisão da taxa

HTML é enviado sem limitação. Os demais objetos são enviados em blocos de até 8192 bytes. A taxa individual é recalculada a cada bloco por $R_c = R_{IP}/N$, com divisão em ponto flutuante. Isso evita truncamento e a imposição indevida de uma taxa mínima de 1 kbps para cada conexão. Após enviar n bytes, a espera nominal é

$$t_{espera} = \frac{8n}{1000 R_c} \quad \text{segundos.} \tag{1}$$

A espera usa `nanosleep`, fora do mutex e em fatias de até 100 ms, permitindo encerramento mesmo com taxas muito baixas. O controle admite uma rajada inicial de um bloco; o tempo de leitura e envio não é descontado da espera. Portanto, não garante uma taxa instantânea rígida em todas as janelas de tempo.

3.3. Painel e estimativas

Uma thread atualiza o painel aproximadamente a cada segundo. Para cada IP, mostra conexões, taxa reservada, taxa por conexão, estimativas e bytes de objetos enviados durante o atendimento atual. A coluna REAL agora usa o total de bytes de todas as conexões daquele IP na janela entre atualizações:

$$V_{IP} = \frac{8\,\Delta B_{IP}}{1000\,\Delta t} \quad \text{kbps}, \tag{2}$$

com Δt em segundos. O contador é reiniciado quando um IP volta a ser admitido após ficar sem conexões.

O intervalo entre a solicitação do HTML e a do primeiro objeto do mesmo IP é usado como estimativa didática de RTT, com resolução de milissegundos e mínimo de 1 ms. O painel mostra também metade desse intervalo e o tamanho do HTML em bits dividido pelo intervalo. Esses números incluem transmissão e processamento do cliente: não medem diretamente RTT TCP, atraso unidirecional ou capacidade do enlace. Navegações simultâneas do mesmo IP podem interferir na associação.

4. Avaliação experimental

4.1. Primeiros experimentos

Os primeiros experimentos foram executados em Zorin OS, com GCC 13.3.0, curl e tcpdump, na interface de loopback, antes da correção da validação e do painel.

Tabela 1. Resultados originais, objeto de 1 MiB por transferência.

Cenário	IP	Status	Vazão útil (kbps)
Isolado	127.0.0.1	200	2001,2
Três IPs	127.0.0.1	200	2003,2
Três IPs	127.0.0.3	200	1503,4
Três IPs	127.0.0.4 (padrão)	200	1004,7
Admissão	127.0.0.2	503	–
Mesmo IP, conexão 1	127.0.0.1	200	1004,3
Mesmo IP, conexão 2	127.0.0.1	200	1004,3

O PCAP dessa execução tem 1647 pacotes, 8 fluxos TCP, 14 requisições GET, 12 respostas 200 e 2 respostas 503. Esse roteiro usava um objeto binário que não é referenciado no HTML; por isso, os experimentos seguintes usam a página com as imagens efetivamente referenciadas.

4.2. Testes automatizados

O servidor foi compilado com C11, otimização O2, Pthreads e opções `-Wall -Wextra -Wpedantic -Werror`. Os testes abrem sockets TCP reais no loopback. A suíte verifica configuração, inicialização, persistência, integridade, admissão, liberação de reservas, taxa padrão, divisão entre conexões e vazão agregada do painel. Os resultados detalhados e logs estão em `evidencias/revisao-v2`.

Os 17 testes foram aprovados. A configuração inválida foi rejeitada antes da abertura do serviço. No teste com imagens, foram recebidos 50.331.864 bytes de conteúdo em 4,6505 s, com vazão útil agregada de 86,58 Mbps. A persistência e os hashes SHA-256 das quatro transferências foram confirmados.

No teste separado de QoS, a taxa padrão de 1000 kbps resultou em 936,2 kbps. Duas conexões do IP com reserva de 2000 kbps atingiram 950,4 e 944,1 kbps. As amostras agregadas do painel com duas conexões ficaram entre 1964,1 e 2097,2 kbps, em torno da reserva total de 2000 kbps. O teste de admissão observou 503 quando não havia reserva disponível e 200 após sua liberação.

O cenário de imagens usa dois clientes concorrentes do mesmo IP, com reserva de 128000 kbps e capacidade de 128000 kbps. Cada cliente solicita HTML, imagem1.bmp e imagem2.bmp na mesma conexão. Cada imagem possui 12.582.966 bytes, verificados por SHA-256. A taxa elevada foi escolhida para testar arquivos completos rapidamente; os testes separados de QoS usam taxas menores.

As medições são observações locais, sem caracterização de capacidade máxima ou intervalos de confiança. A vazão útil do lote usa bytes de corpo recebidos e tempo total até a conclusão dos dois clientes. Ela difere da vazão da interface, que pode incluir cabeçalhos, retransmissões e tráfego de fundo.

4.3. Avaliação com Wireshark e IPTraf

O servidor foi executado com `./server 8095 www 6000 taxas.txt` em um notebook da equipe (Ubuntu 24.04, kernel 7.0, x86_64), na interface de loopback, em 19/10/2026. As taxas cadastradas foram 2000 kbps para 127.0.0.1, 3000 kbps para 127.0.0.2 e 1500 kbps para 127.0.0.3; o endereço 127.0.0.4 não consta do arquivo e recebe a taxa padrão. Cada cliente `curl`, ligado a um endereço de origem com `–interface`, solicitou o HTML e depois uma imagem de 12.582.966 bytes na mesma conexão. O tráfego da porta 8095 foi capturado com o `dumpcap`, o mecanismo de captura do Wireshark, com 128 bytes por pacote, suficientes para os cabeçalhos TCP e as linhas HTTP. A captura pode ser aberta no Wireshark; os números abaixo foram extraídos dela por um roteiro em Python incluído nas evidências. Durante a execução, o IPTraf-ng monitorou os fluxos TCP da interface.

Foram executados quatro cenários: (A) 127.0.0.1, 127.0.0.3 e 127.0.0.4 baixam uma imagem simultaneamente; (B) aos 3 s, 127.0.0.2 solicita admissão quando 4500 dos 6000 kbps já estão reservados; (C) duas conexões simultâneas de 127.0.0.1 baixam uma imagem cada; (D) 127.0.0.2 tenta novamente com a reserva liberada. A Tabela 2 compara a taxa configurada com a vazão de payload TCP medida na captura, do pedido da imagem ao último segmento, e com a vazão útil informada pelo `curl`.

Tabela 2. Vazão por IP medida no Wireshark e no cliente.

Cenário	IP	Taxa (kbps)	Wireshark (kbps)	curl (kbps)	HTTP
A	127.0.0.1	2000	1988,3	1987,6	200
A	127.0.0.3	1500	1493,0	1492,6	200
A	127.0.0.4 (padrão)	1000	996,4	996,2	200
B	127.0.0.2	3000	–	–	503
C	127.0.0.1, conexão 1	2000/2	994,4	994,3	200
C	127.0.0.1, conexão 2	2000/2	994,4	994,3	200
D	127.0.0.2	3000	–	–	200

A Figura 1 mostra a vazão a cada segundo, obtida da captura. Cada IP permanece próximo da própria taxa até o fim da transferência. No cenário C, as duas conexões de 127.0.0.1 ficam em torno de 1000 kbps cada e a soma volta a cerca de 2000 kbps, o que confirma a divisão da taxa por endereço. As oscilações de um segundo se devem aos blocos de 8 KiB e ao limite de uma janela de amostragem cair no meio de um bloco.

A captura tem 15.465 pacotes em 202,3 s, sem descartes, e 7 conexões TCP, todas com *handshake* completo. Nela aparecem 12 requisições GET, 11 respostas 200 e uma resposta 503. Nenhum segmento de dados foi retransmitido. Em cada conexão, o GET da imagem aparece no mesmo fluxo TCP logo após a resposta ao HTML, o que confirma a persistência. A única conexão encerrada com RST é a recusada no cenário B.

No IPTraf-ng, durante o cenário A, o fluxo de 127.0.0.1:8095 para 127.0.0.3:46050 apareceu com taxa de 2965,4 kbps, cerca do dobro dos 1500 kbps configurados. Isso ocorre porque, no loopback, o mesmo pacote passa pela interface na saída e na entrada e é contabilizado duas vezes. Dividido por dois, o valor (1482,7 kbps) fica próximo do medido na captura. A mesma tela mostra a proporção esperada entre os fluxos: 5.820.264 bytes para 127.0.0.3 e 3.882.924 bytes para 127.0.0.4, razão de 1,50, igual

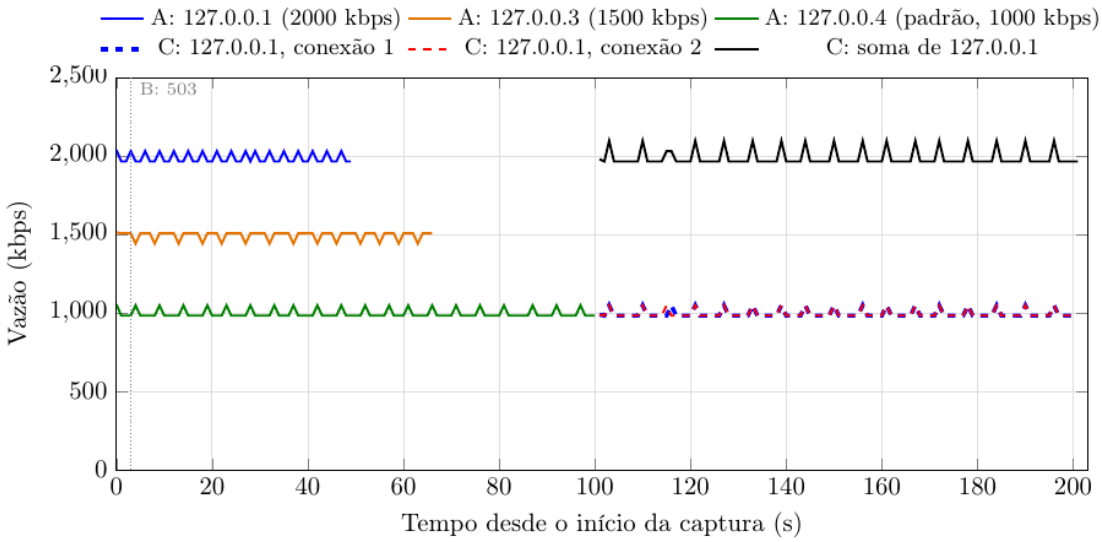


Figura 1. Vazão do servidor para cada IP, a cada 1 s, calculada a partir da captura.

à razão entre as taxas de 1500 e 1000 kbps. O IPTraf também mostra tráfego da interface que não pertence ao experimento, como consultas DNS locais.

4.4. Limitações

Os endereços 127.0.0.x simulam clientes distintos, mas não são hosts diferentes, e o loopback não reproduz Ethernet nem Wi-Fi: não há perdas nem disputa pelo meio. O intervalo entre a resposta ao HTML e a requisição da imagem foi de cerca de 0,2 ms. Como o painel mede em milissegundos com mínimo de 1 ms, ele exibe RTT de 1 ms, e a banda estimada (532 bytes divididos por 1 ms, 4256 kbps) não representa o cliente. Não foi emulado atraso com `tc netem`. Os cenários em redes físicas, preferenciais no enunciado, não foram realizados.

5. Conclusões e próximos passos

A versão corrigida implementa os mecanismos centrais de QoS e corrige falhas de validação e de apresentação da vazão por IP. Os testes com arquivos completos acrescentam evidência de persistência e integridade das imagens da página. As taxas configuradas são reservas e alvos de envio de objetos; HTML, cabeçalhos e erros não entram no limitador e não se afirma um teto físico absoluto. Conexões ociosas participam da divisão enquanto permanecerem abertas.

A avaliação com Wireshark e IPTraf-ng confirmou o comportamento esperado do limitador: vazão por IP a menos de 1% da taxa configurada, divisão igual entre duas conexões do mesmo endereço, nenhuma retransmissão TCP e recusa por 503 quando a reserva excede a capacidade. As estimativas de RTT e de banda do painel, porém, não são informativas no loopback, onde o intervalo real é de cerca de 0,2 ms. Como próximos passos, pretendemos medir esses intervalos em microssegundos, usar prazos de envio cumulativos, separar sessões de navegação do mesmo IP e repetir os cenários em Ethernet e Wi-Fi com hosts distintos e atraso emulado por `tc netem`.

6. Apoio de IA e disponibilidade

Tivemos auxílio de IA (OpenAI ChatGPT e Anthropic Claude) [5] no relatório e na correção dos códigos, principalmente dos testes automáticos. Esse apoio também está registrado nos comentários do código e nos arquivos README. Código, evidências e roteiros estão disponíveis em `https://github.com/giovanapiazza/Redes_De_Comp`.

Referências

[1] Fundamentos e Avaliação de Redes de Computadores (2026). Trabalho de programação em rede: especificação do trabalho prático. 2026/2.

[2] Forouzan, B. A.; Mosharraf, F. (2013). Redes de computadores: uma abordagem top-down. AMGH.

[3] Fielding, R.; Nottingham, M.; Reschke, J. (2022). HTTP/1.1. RFC 9112. `https://www.rfc-editor.org/rfc/rfc9112`.

[4] Fielding, R.; Nottingham, M.; Reschke, J. (2022). HTTP Semantics. RFC 9110. `https://www.rfc-editor.org/rfc/rfc9110`.

[5] OpenAI (2026). ChatGPT. `https://chatgpt.com/`. Anthropic (2026). Claude. `https://claude.ai/`.